SnapCare pill dispenser

Building one, from a bare Raspberry Pi

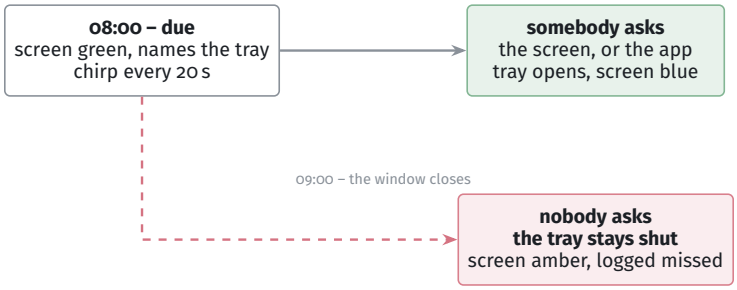
Raspberry Pi 3B+ • 64-bit Raspberry Pi OS (Trixie) • eight stages

1 A dispenser that asks first

Read this before you wire anything

Nothing drops by itself. At the scheduled minute the machine *offers* the dose: the screen turns green and names the tray, and the buzzer chirps every 20 seconds. The tray opens only when somebody asks — a tap on the screen, or release in the app. They do the same thing. Ask within the hour or the screen goes amber, the dose is logged missed, and the tray never opened.

A machine sitting there chirping is working, not broken.



The window is one hour, and it is `GRACE_MINUTES` if you want it to be something else. Everything after this page is wiring.

Pin numbers are given as *GPIO n (pin m)*: the first is what the software calls it, the second is what you count on the board. Where this guide and `snapcare_dispenser/config.py` disagree, the file is right and this page is stale.

2 What to buy

Qty	Part	Notes
1	Raspberry Pi 3B+	Any Pi with the 40-pin header; the pin map assumes a 3B/3B+
1	microSD, 8 GB+	64-bit Raspberry Pi OS, Trixie or newer
1	Pi power supply, 5 V 2.5 A	The official one. Servos do **not** come off this
1	5 V supply, 2 A+	Separate, for the servos alone
1	1000 µF electrolytic, 10 V+	Across the servo rail, close to the servos
1 per tray	SG90 micro servo	Up to eight. Adding one later is a plug
1	Passive piezo buzzer	**Passive**, not an active beeper. They look identical in the bin; Stage 2 tells them apart by ear
1	100 Ω resistor	In series with the buzzer
1	Momentary push button	*Optional.* The touch screen is the control; fit this only for a second, physical one. Any normally-open switch, two wires, no resistor
1	DS1302 RTC module	The five-pin board with a coin-cell holder
1	CR2032 cell	**Not** the LIR2032 the holder is often marked for. The trickle charger is never enabled, so a rechargeable arrives flat and stays flat
The screen		
1	ILI9341 2.8 in SPI LCD, capacitive touch	240×320. The panel is four-line SPI; the touch controller is a separate chip on I^2C, and its four pins are the ones marked `CTP_`. **Check yours before powering it**: boards with an on-board regulator want 5 V, bare ones are 3.3 V and are destroyed by 5 V. See §7
—	Jumper wire, terminal blocks	Dupont leads for the bench; screw terminals for anything that gets carried

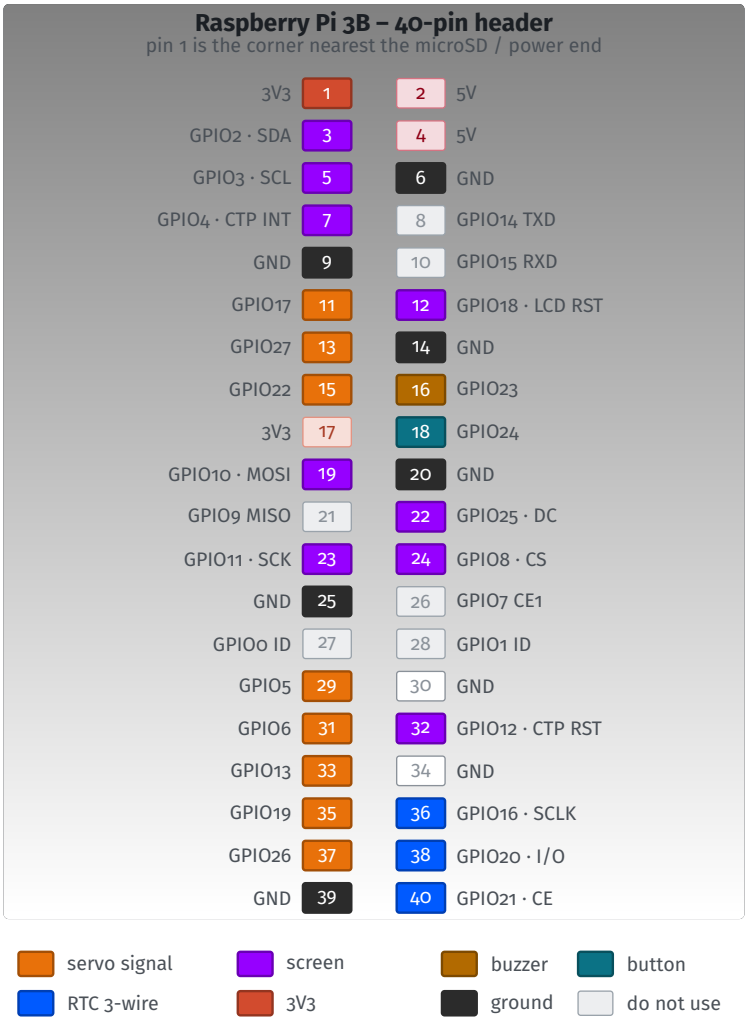
Two supplies, and they are not interchangeable

The Pi has its own. The servos and nothing else share the second one. A stalled SG90 pulls most of an amp; a Pi that browns out mid-write corrupts its SD card, and the file being written is the one recording whether a dose has already been given.

The 3.3 V pin now carries three things — the clock chip, the screen, and the LCD's backlight, which is 50–100 mA on its own. That is within budget, and it is the first thing to check if the clock starts behaving oddly after you fit the screen.

Unplug means unplug. The GPIO pins are live whenever the board is, and `sudo halt` does not change that. Power off to wire, power on to test. The servo supply is a separate plug and stays off until Stage 6.

3 The header, and every pin this build touches



GPIO 2 and 3 carry permanent 1.8 kΩ pull-ups that cannot be switched off. That is why no servo goes near them — a signal line idling high before the software starts slams a horn to one end with a loaded tray on it — and it is exactly what makes them the I²C pair, which wants those pull-ups and finds them already fitted. The screen's touch controller is what uses them here, and a DS3231 build would share the same two pins. Run `pinout` for the whole header.

	Signal	GPIO	Pin	Notes
	Servo, trays 1–3	17, 27, 22	11, 13, 15	signal only — **not** power
	Servo, trays 4–8	5, 6, 13, 19, 26	29, 31, 33, 35, 37	fit a hopper and it works
	Piezo buzzer	23	16	through 100 Ω
	Take-dose button	24	18	optional — the screen is the button
	LCD SDI (MOSI), SCK, CS	10, 11, 8	19, 23, 24	
	LCD RS (DC), RESET	25, 18	22, 12	the module marks DC `LCD_RS`
	Touch SDA, SCL	2, 3	3, 5	I^2C — the `CTP_` pins
	Touch RST, INT	12, 4	32, 7	the kernel driver owns both
	DS1302 CLK, DAT, RST	16, 20, 21	36, 38, 40	bit-banged; no bus, no kernel driver
	3.3 V	—	1, 17	clock chip, screen, backlight
	Ground	—	6, 9, 14, 20, 25, 39	one of these must reach the servo supply's ground
	do not use	0, 1, 7, 9, 14, 15		HAT ID EEPROM, the rest of SPI, the serial console
	free	—	—	the touch controller took the last two

4 Stage 0 — the card and the install

Flash 64-bit Raspberry Pi OS and put the Pi on the network **with an Ethernet cable**. The installer needs a network, and the machine's own way onto WiFi does not exist until the installer has run. The cable is a build tool, not part of the machine — it comes out at Stage 7.

```
git clone <this repo> && cd h1-SnapCare/pill_dispenser/firmware
sudo ./service/install.sh
sudo reboot
```

It sets the hostname to `pilldispenser`, turns on I²C and SPI for the screen, installs the service, and prints this device's permanent setup network. **Keep what it prints** — that is Stage 7.

Two commands are used at every stage below. They are written out here and referred to by name afterwards.

```
# "status"
curl -s localhost/api/status | python3 -m json.tool

# "selftest" - asks you what you can see and hear, then writes a report
sudo systemctl stop snapcare-dispenser
sudo snapcare-dispenser --selftest
sudo systemctl start snapcare-dispenser
```

Prove it

`status` answers, and `gpio.missing` lists everything not yet fitted. Every stage below is one item leaving that list.

Three things about the install

`apt install pigpio` fails on Trixie and is not coming back. `python3-lgpio` is preinstalled and is the driver. Nothing to do — but the error is alarming if you go looking for it.

If the installer stops on `could not get lock /var/lib/dpkg/lock-frontend`, something else is mid-update. It waits five minutes and then gives up; wait and run it again. **Do not delete the lock file.** It does not release the lock and it breaks the package database.

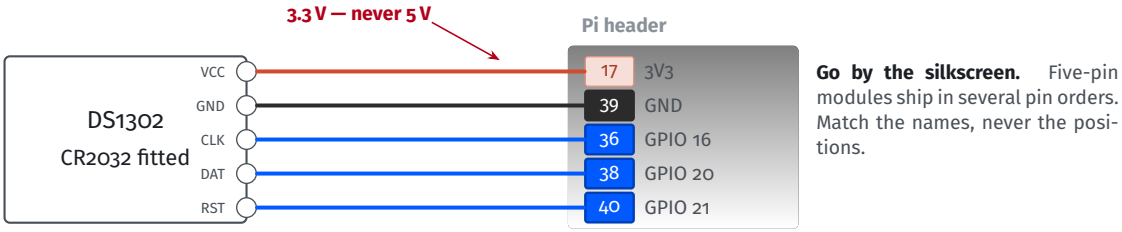
`selftest` reports `skip` for anything nobody answered. "Everything passed" and "everything anybody looked at passed" are different sentences.

Everything except the hardware runs on a laptop:

```
make            # the whole test suite, no board
make run        # the firmware with mock servos, on port 8099
make -C .. demo # the same, plus a relay and a browser app
```

5 Stage 1 — the clock chip

A Pi has no clock of its own. Without this chip a dispenser that loses power overnight comes back believing it is yesterday and opens the wrong day's compartment, looking perfectly healthy while it does. **Fit the cell before you set the time.**



Go by the silkscreen. Five-pin modules ship in several pin orders. Match the names, never the positions.

The DS1302's data line drives back into the Pi, and that pin is not 5 V tolerant. The chip runs happily at either voltage, so nothing complains until the GPIO is dead.

Prove it

Set the clock, then take the power away for real:

```
curl -X POST http://localhost/api/clock \
     -H 'Content-Type: application/json' -d "{\"iso\":\"$(date -Is)\"}"
```

Pull the power lead. Wait a minute. Plug it back in with the Ethernet cable out, and read `status`: `clock.trusted` must be `true` and `clock.source` must read `rtc`.

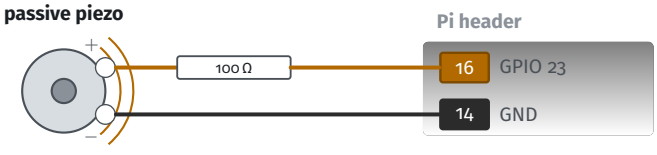
`sudo shutdown now` does not test this

A halt leaves the 3.3 V rail live, so the chip never runs on its cell and you have tested nothing. Pull the plug. The DS1302's halt bit is a control and not a status — the chip cannot tell you whether it kept time — so this is the only evidence there is.

`i2cdetect` and `/dev/rtc0` have nothing to say about this chip. It has no bus and no kernel driver. Read `status`.

6 Stage 2 — the buzzer

The chirp is the prompt, not an announcement. A machine that cannot make it waits out the hour in silence and records a miss.



Prove it

`selftest` sounds three chirps and asks what you heard.

A clear tone is a pass. **Three faint clicks** is an active buzzer: it has its own oscillator, so it makes a noise at a frequency nobody chose. Fit a passive element instead.

(`POST /api/drop` deliberately stays silent — a manual drop is somebody checking a hopper, not

> a dose anybody has to be fetched for.)

7 Stage 3 — the screen

The screen is most of what the person in the room ever sees, and it is the one part that can say *which tray* and *what time*.

It is also the only *control* this build has: there is no physical button, and a tap anywhere on the glass releases a waiting dose. With no screen fitted the machine still doses and still chirps, and the app is the only way to release.

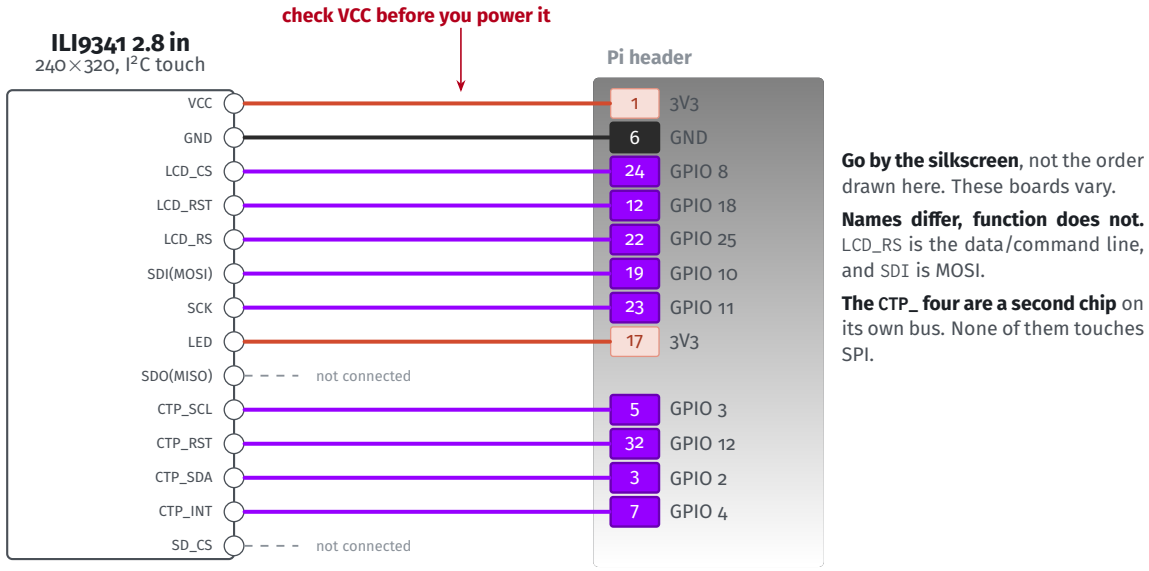
The module carries two chips that share nothing but the connector. The panel is on SPI; the touch controller is on I²C, and its pins are the four marked `CTP_`. Wire them as two separate jobs.

Module pin	Goes to	Notes
The panel — SPI		
VCC	*read your module*	3.3 V or 5 V. Getting this wrong is the one mistake that ends the board
GND	GND (pin 6)	
LCD_CS	GPIO 8 (pin 24)	CE0
LCD_RST	GPIO 18 (pin 12)	
LCD_RS	GPIO 25 (pin 22)	*register select* is the data/command line. The firmware calls it DC
SDI(MOSI)	GPIO 10 (pin 19)	
SCK	GPIO 11 (pin 23)	
LED	3.3 V (pin 17)	backlight, always on
SDO(MISO)	**nothing**	the panel is written and never read
The touch controller — I²C		
CTP_SCL	GPIO 3 (pin 5)	
CTP_SDA	GPIO 2 (pin 3)	
CTP_RST	GPIO 12 (pin 32)	
CTP_INT	GPIO 4 (pin 7)	finger-down; the kernel driver owns it
Neither		
SD_CS	**nothing**	the card slot on the back of the module. This build stores nothing on it

Meter CTP_SCL and CTP_SDA before either touches the header

GPIO 2 and 3 are not 5 V tolerant, and on a module whose VCC is 5 V the touch pull-ups can sit on the 5 V rail. Power the module from its VCC and ground only, leave the other twelve pins in the air, and measure `CTP_SCL` to ground: **above 3.6 V, those two wires do not go on**. The panel half is unaffected — those lines are driven by the Pi, not back into it.

It reads about 3.3 V on every board that runs its touch chip off the internal regulator, which is most of them. It costs thirty seconds to know which you have.



The panel needs nothing else; the touch half needs one line. `install.sh` already turned on SPI and I²C, so the picture works as soon as it is wired. The touch controller does not: the kernel has to be told which chip is on the bus, and until it is, `/dev/i2c-1` has a device on it that nothing is driving — the screen paints perfectly and a tap does nothing.

Ask the bus what is there:

```
sudo i2cdetect -y 1
```

One address answers, and it names the part:

Address	Part	Overlay
`0x38`	FT6206 / FT6236	`edt-ft5406`
`0x5d` or `0x14`	GT911	`goodix`
`0x48`	NS2009	`ns2009`

Overlays take their pin numbers as parameters and the names differ between them, so read the one you need before you write the line — `dtoverlay -h goodix` prints its parameters, and `dtoverlay -a` lists every overlay this kernel has. Both `CTP_INT` (GPIO 4) and `CTP_RST` (GPIO 12) go in it. Then add the line to `/boot/firmware/config.txt` and reboot.

Delete the `ads7846` line the installer wrote

`install.sh` adds `dtoverlay=ads7846` to `/boot/firmware/config.txt`. That is a driver for a resistive controller on SPI, and this module does not have one. Take the line out.

Leaving it is worse than useless. The driver can bind and publish an input device for a chip that is not there, and the firmware takes the first touch device it finds — so it reads the empty one, reports a touch panel fitted, and no tap ever releases a dose. A screen that says it is working and a tray that never opens is exactly the failure this guide exists to prevent.

Prove it

After the reboot, one of these must name your controller:

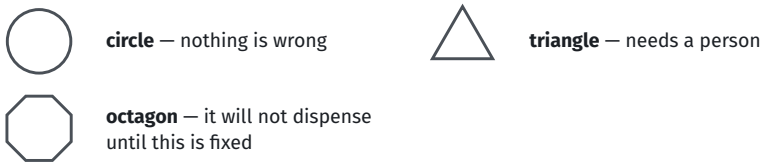
```
cat /sys/class/input/event*/device/name
```

Nothing listed means the overlay did not bind: wrong overlay, or wrong pin numbers in it. A build with no touch is a working build — the chirp still fetches somebody and the app still releases the dose — but the glass is then not a control, and Stage 4 is no longer optional.

If the device is there and a tap still does nothing, the firmware did not recognise its name. Point it straight at the node:

```
sudo systemctl edit snapcare-dispenser
# add: Environment=SNAPCARE_TOUCH_DEVICE=/dev/input/event0
sudo systemctl restart snapcare-dispenser
```

Colour is only half of it. Roughly one man in twelve cannot use the red-against-green difference at all — so every screen also carries a *shape*, filled rather than outlined so it is told apart by its mass, and the shape means the same thing to everybody:



The octagon is rare on purpose. A warning somebody sees three times a day is a warning they stop reading.

The frame is three bands, and they never move. A status bar across the top carries seven fixed slots — wifi, relay, clock, tray, power, coin cell, SD card — each an icon with a solid severity shape under it, plus the wall clock. It never reorders and never reflows, because a row you cannot learn is a row you have to read. An empty field draws a dim icon and *no* mark: absence is not a fault and is certainly not a tick.

The middle band is the one dose that matters, with the time as the largest thing on the panel and the tray number under the shape. Below it, a strip carries the whole day: one solid mark per dose with its time under it, and a white divider showing where you are. Taken is a blue circle, waiting a green circle, missed an amber triangle, still to come a steel square — so “did I take the eight o’clock one” is answered by looking.

The device’s IP address sits permanently under the time. That matters more than it sounds: Chrome on Android cannot resolve `pilldispenser.local`, so those four numbers are the only way in for about half of everybody who tries — and the screen that used to show them was part of a rotation that dropped it exactly when there was an address to show.

A tap anywhere releases a dose. There are no on-screen buttons and nothing to aim at, deliberately: a target can be missed, and this is a machine for hands that shake. Touching it also brings the panel out of its night colours for half a minute.

Prove it

If the glass is dark, walk it one stage at a time:

```
sudo snapcare-dispenser --display-test
```

It lights the backlight, floods red then green then blue, draws a windowed rectangle and finally a real frame, asking after each what you can actually see — then says what the first failure means. `-display-hz 8000000` retries on a slower bus, which is the first thing to try on a speckled panel.
Then `selftest`. It paints each state in turn and asks which one you are looking at, **what shape is on it**, and what colour. Those questions are the whole test; answer them honestly.

A lit screen is not proof, and one wrong bit swaps two doses

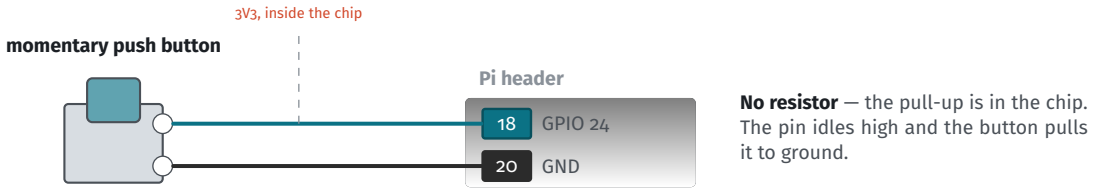
The LCD is write-only. A dead panel, an unplugged one and one wired a row off all accept exactly the same writes without complaint, and the backlight comes on regardless of whether anything is being drawn.

If the panel's BGR bit is wrong, red and blue trade places — so *released* and *missed* swap, and the machine tells somebody their pills are in the cup an hour after the window closed. That is why the self-test asks what colour you can see and not whether you can see anything.

8 Stage 4 — the take-dose button (optional)

The screen is already the control: a tap anywhere on the glass releases a waiting dose, and Stage 3 fitted it. This stage adds a *second*, physical control, and it is worth two wires in exactly two situations — a person who cannot reliably work a touch screen, and a bench where you want to prove the release path without a finger on the glass.

Skip it otherwise. Both inputs reach the same `release()`, so nothing about the machine's behaviour changes either way.



It releases a due dose. It does not dispense

Pressed when a dose is due, the tray sweeps. Pressed at any other time, **nothing happens** — no tray moves and nothing is logged.

That is what stops a hopper being walked empty one press at a time by somebody curious. Every one of those pills would be missing from a dose later in the week, with nothing anywhere saying so.

Prove it

`selftest` asks you to press it once and reports what it counted.

1 press seen	pass
nothing seen	open circuit — check the wire from pin 18 to pin 20
4 presses seen	the debounce filter is not running

Then start the service and press it with no dose due:

```
journalctl -u snapcare-dispenser -f
```

It must log `button pressed with no dose waiting` and open nothing. If a tray moves here, stop and fix it.

A build with no button is the normal build: the screen and the app both release doses without it.

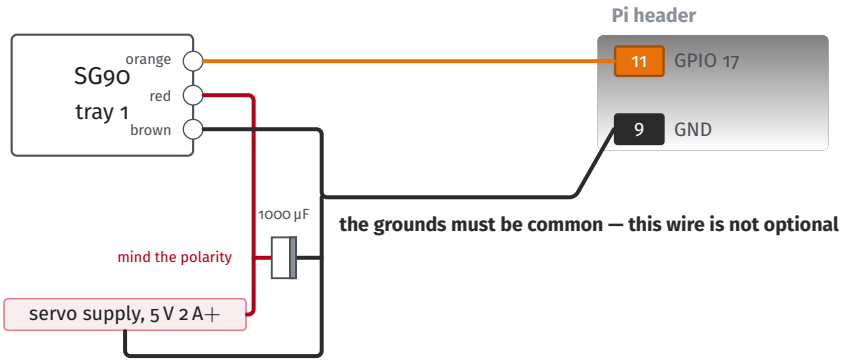
9 Stage 5 — the servo rail, before any servo

Wire the second supply and measure it with **no servos attached and no signal wires to the Pi**. This is where a 12 V brick or a reversed lead is caught, while there is nothing connected for it to destroy.

Prove it

Meter across the rail with nothing attached: **5 V ±0.25 V, steady**.

10 Stage 6 — the servos



Servos that twitch or ignore commands entirely read as a firmware fault and are not one. It is this wire.

The other trays follow the same pattern, one signal wire each:

Tray	1	2	3	4	5	6	7	8
GPIO	17	27	22	5	6	13	19	26
Pin	11	13	15	29	31	33	35	37

Prove it

One tray at a time, and watch which hopper moves:

```
sudo snapcare-dispenser --drop 1     # then 2, then 3, six seconds apart
```

“A tray moved” is the wrong question

Two trays wired to each other’s pins both move, so a check that asks whether something happened passes. The machine then gives the right number of pills from the wrong compartment, and nobody notices until the wrong medication has been taken for a week. Ask *which one moved*.

Fit the horns with the servo parked. Run `--drop` once and let it finish; the horn is then at rest. Nothing on an SG90 reports its position, so a horn fitted while the servo happens to be at the far end is a tray that never opens — and the dose is logged as delivered anyway.

A servo buzzing at the end of its stroke is being driven past a stop it has already reached, and that one burns out over weeks. Trim that tray alone, in microseconds:

```
# snapcare_dispenser/config.py
SERVO_TRIM = {3: (520, 2450)}
```

All eight trays are accepted whether or not they exist

An SG90 reports nothing, so the firmware cannot tell a missing hopper from a fitted one. A schedule naming a tray you never built is accepted, never dispensed, and reported by nothing. Say how many you have:

```
sudo systemctl edit snapcare-dispenser    # add: Environment=SNAPCARE_TRAYS=3
sudo systemctl restart snapcare-dispenser
```

11 Stage 7 — WiFi, by scanning a code

Unplug the Ethernet cable. From here the machine joins a network the way its owner will do it: by scanning a printed code. No keyboard, no monitor.

```
sudo snapcare-dispenser --show-qr     # the permanent setup network
sudo snapcare-dispenser --show-label  # everything the printed label must carry
```

1. Print the code. It is derived from this board and **never changes**, so it is printed once for the life of the device.
2. Scan it with a phone. The phone joins the dispenser's own network.
3. A setup page opens by itself. Pick the house network and type its password.
4. The dispenser joins it, its own network disappears, and the phone goes back to the house WiFi.

Prove it

From the phone, on the house WiFi: `http://pilldispenser.local/api/status` answers.

If that name does not resolve — Chrome on Android does no mDNS — use the IP address the setup page showed you.

The pairing code on the label is a credential

Anyone who can read it can send this device a schedule *and release a dose*. Put the label where the owner will find it and a visitor will not: inside a battery door, or on the underside.

The QR carries your relay's address alongside the code, so reprint it if you ever move relays. The code itself comes from the board and never changes.

12 Commissioning — six tests a bench cannot run

`selftest` checks everything a program can check. These six it cannot, and every one of them is a failure that looks exactly like a healthy machine.

Test	What to do, and what must happen
The power cut	Pull the power at 08:30, after the 08:00 dose has been released, and restore it. No second dose, and the log survives. Without persisted state this dispenses twice, and no demo ever shows it
The coin cell	Unplug the Pi, leave it a minute, bring it back with the network unplugged. The clock must still be trusted and `clock.source` must read `rtc`. Nothing on a bench with WiFi proves this, because NTP vouches first
The wrong password	Type the WiFi password wrong on purpose at Stage 7. The setup network must come back within seconds and say the password was refused. On a machine joined by scanning a code, this is the difference between a second try and re-imaging the card
The internet cut	Pull the router's WAN cable, leave the phone and the dispenser on the LAN, and release a due dose from the app. The pill must still come out. This catches a release path that only works through the relay
The dose nobody takes	Let a scheduled dose sit for its whole hour without pressing anything. The tray must stay shut, the screen must go amber, and the log must record it missed. A build that quietly drops it at the end has lost the point of asking first
The idle button	Press the button with nothing due. Nothing must move and nothing must be logged

To make a dose fall due on demand, send a schedule for two minutes from now:

```
curl "http://pilldispenser.local/schedule?payload=SC1*W*1:0800-127-000"
```

`SC1*W*`*tray*`:`*HHMM*`-`*days*`-`*length*. Several times on one tray join with `.`, several trays with `/`. *days* is a bitmask with Monday as 1 — `127` is every day, `021` is Mon/Wed/Fri. *length* is a number of days, `000` for no end date.

13 When it does not work

Symptom	Look at
Doses	
The screen goes green and chirps, but no tray ever opens	Working as designed. Something has to ask: the button, or release in the app
Pressing the button does nothing	`gpio.button.presses` in `status`. Going up means the button works and no dose was due. Staying at zero means the wire from pin 18 to pin 20
Nothing is dispensed at all	`device.blockedReason`. It is one of two things and they need opposite fixes: an untrusted clock, or no GPIO library
Doses expire untaken every day	Nobody is being told. Check `gpio.missing` for the buzzer
A tray in the schedule never dispenses	It was accepted but never fitted. Set `SNAPCARE_TRAYS`
Screen	
Nothing on the screen, ever	`gpio.display` in `status`. `fitted:false` carries the reason
Lit, but nothing is ever drawn	Very likely logic levels. A 5 V module with a plain 74HC245 shifter needs ~3.5 V to read a high and the Pi puts out 3.3 V, so every line reads low. Power VCC from 3V3 instead: a 3.3 V board works, a 5 V one goes dark, and that tells you which you have
Backlight dark, not the picture	LED is on a GPIO nobody drives. Set `SNAPCARE_DISPLAY_BACKLIGHT` to its BCM pin; until then it sits low
A tap on the screen does nothing	No touch overlay in `/boot/firmware/config.txt`, or no reboot since. The log says so at startup. `i2cdetect -y 1` first: no address means the four `CTP_` wires, not the overlay
A tap does nothing, and `ads7846` is in the input device names	The installer's line is still there, and the firmware bound to it instead of your panel. Delete `dtoverlay=ads7846` and reboot
The touch device is listed and taps still do nothing	The firmware does not know that name. Set `SNAPCARE_TOUCH_DEVICE` to the event node
LCD: lit but blank	The backlight is independent of the data lines, so a lit panel proves only that LED has power. Check LCD_RS (pin 22) and LCD_RST (pin 12)
LCD: the colours are wrong	The BGR bit. *Released* and *missed* swap when it is wrong; do not ship it like this
LCD: mirrored or rotated	`DISPLAY_ROTATION` in `config.py`
Servos and the clock	
Servo twitches or ignores commands	The common ground. It is not optional
Servo hums between doses	`gpio.servo.timing` — `python` is the slow fallback; `sudo apt install python3-lgpio`
Pi reboots when a servo moves	Servos on the Pi's 5 V pin, or no bulk capacitor
Clock right at shutdown, wrong at boot	The cell is dead or not seated. `clock.rtc.detail` says which
`clock.rtc` says "no record of ever having been set"	The chip has never been written. Set the clock once, then pull the power to check it stuck
The RTC never changes after a `shutdown` and a boot	A shutdown leaves the board powered, so nothing was tested. Pull the plug
Network and install	

Symptom	Look at
`pilldispenser.local` does not resolve	Android does no mDNS. Use the IP address
Setup page never leaves “waiting…”	The phone has not gone back to the house WiFi. Switch it over
Setup page says no address was handed out	The password was right and DHCP was not. Usually a router that needs restarting
A schedule request answers 421	The `Host` header is not a name this device answers to. Use `pilldispenser.local` or the IP
`install.sh` stops on a dpkg lock	Something else is mid-update. Wait, run it again, and do not delete the lock file

14 Appendix — cloud sync, if you want it

Without this the dispenser is LAN-only, which is a complete machine: the schedule arrives over the house WiFi or off a photographed paper sheet, and the SD card and the clock chip keep the alarms running.

With it, a schedule edited in the app reaches the device from anywhere, and a carer in another country can release a dose. Deploy one relay for all your devices — see `../cloud_relay/` — then:

```
sudo systemctl edit snapcare-dispenser
# add: Environment=SNAPCARE_RELAY_URL=https://your-relay.example
sudo systemctl restart snapcare-dispenser
```

Prove it

Pair the app with the code on the label. Then pull the router’s WAN cable and release a due dose from a phone on the same LAN: the pill still comes out. The relay is a way in, never a dependency.